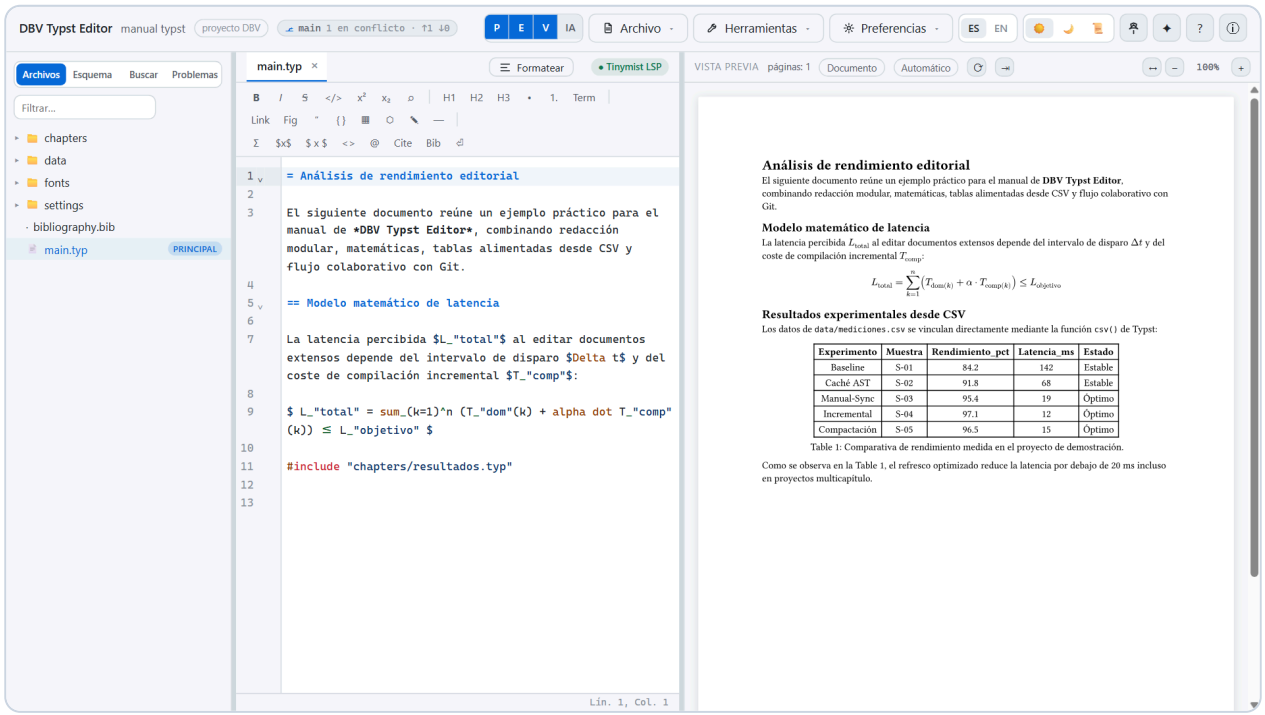
Manual de Usuario · Español · v0.15.0

DBV Typst Editor

Manual completo de usuario: redacción académica y técnica, menús, colaboración con Git, herramientas visuales, asistentes e IA opcional



Autor: David Bueno Vallejo
Licencia: MIT License
Typst: 0.15.1 (embebido / bundled)
2026

Repositorio: github.com/davidbuenov/dbv-typst-editor · Web

Contenido del manual

1. Qué es DBV Typst Editor 1
   1.1. Novedades principales de la versión v0.15.0 1
2. Primeros pasos y lanzador inicial 2
3. Espacio de trabajo y los cuatro paneles 4
4. Referencia completa de los menús 5
   4.1. Menú Archivo 5
   4.2. Menú Herramientas 5
   4.3. Menú Preferencias 6
5. Edición de alto rendimiento, barra de herramientas y modo Vim 8
   5.1. Barra de herramientas contextual y citas BibTeX 8
   5.2. Modo Vim opcional, números relativos y refresco manual 8
6. Personalización visual: temas, colores y contraste WCAG 10
7. Trabajo colaborativo con Git 11
   7.1. Clonar e iniciar un proyecto compartido 11
   7.2. Estado remoto y popover de Git en cabecera 11
   7.3. Resolvedor visual de conflictos bloque a bloque y cambios en disco 12
8. Herramientas visuales y asistentes con ejemplos prácticos 13
   8.1. Editor de diagramas (`cetz`) 13
   8.2. Editor visual de ecuaciones y pegado desde LaTeX (`MiTeX`) 13
   8.3. Diagramas de secuencia (`chronos`) 14
   8.4. Diagramas de Gantt (`gantty`) 14
   8.5. Tableros Kanban (`kantan`) 14
   8.6. DOT / Graphviz (`diagraph`) 14
   8.7. Visor de datos CSV / TSV y tablas dinámicas (`csv()`) 14
   8.8. Ejecutar scripts Python y lógica JavaScript embebida (`jogs`) 15
9. Snippets de VS Code, Sublime Text y atajos personalizados 16
10. Fuentes propias, Typst Universe y Terminal avanzado 17
    10.1. Fuentes propias en `fonts/` 17
    10.2. Typst Universe: plantillas y paquetes 17
    10.3. Terminal integrado de Typst y servidor MCP 18
11. Guía completa del Asistente de IA (Local, Agentes o Nube) 19
    11.1. Vía 1: IA local en tu equipo (privacidad total) 19
    11.2. Vía 2: Proveedores en la nube con clave de API 20
    11.3. Vía 3: Agentes CLI con tu suscripción (protocolo ACP) 20
    11.4. Qué puede hacer la IA sobre tu proyecto y propuestas verificadas 21
    11.5. Privacidad y solución rápida de problemas de IA 21
    11.6. El servidor MCP de DBV para otros agentes (`Herramientas › Servidor MCP…`) 21
12. Créditos y agradecimientos 23
    12.1. Autoría y colaboradores 23
    12.2. Agradecimientos de la comunidad (`v0.15.0`) 23
    12.3. Proyectos de código abierto que hacen posible DBV 23

1.

Qué es DBV Typst Editor

DBV Typst Editor es un entorno de escritorio ligero y autocontenido (Rust + Tauri v2, sin Electron) diseñado específicamente para redactar trabajos fin de grado (TFG), tesis doctorales, artículos científicos, informes técnicos, presentaciones y currículums con el sistema de composición **Typst**.

A diferencia de un editor de código genérico, la aplicación integra el compilador oficial de Typst en memoria junto al servidor de lenguaje **Tinymist LSP**: escribes a la izquierda y ves las páginas compuestas a la derecha en tiempo real, con diagnóstico inmediato de errores, gestión de citas BibTeX, asistentes visuales y exportación a PDF, PNG o archivo de proyecto portable `.dbvt`.

> **Sin ataduras y 100 % compatible**
>
> Todo proyecto creado con DBV Typst Editor es un proyecto Typst estándar. Puedes compilarlo con el binario oficial `typst compile main.typ` desde cualquier terminal, compartirlo por Git con usuarios de otros editores o empaquetarlo completo en un archivo `.dbvt` con sus fuentes propias incluidas.

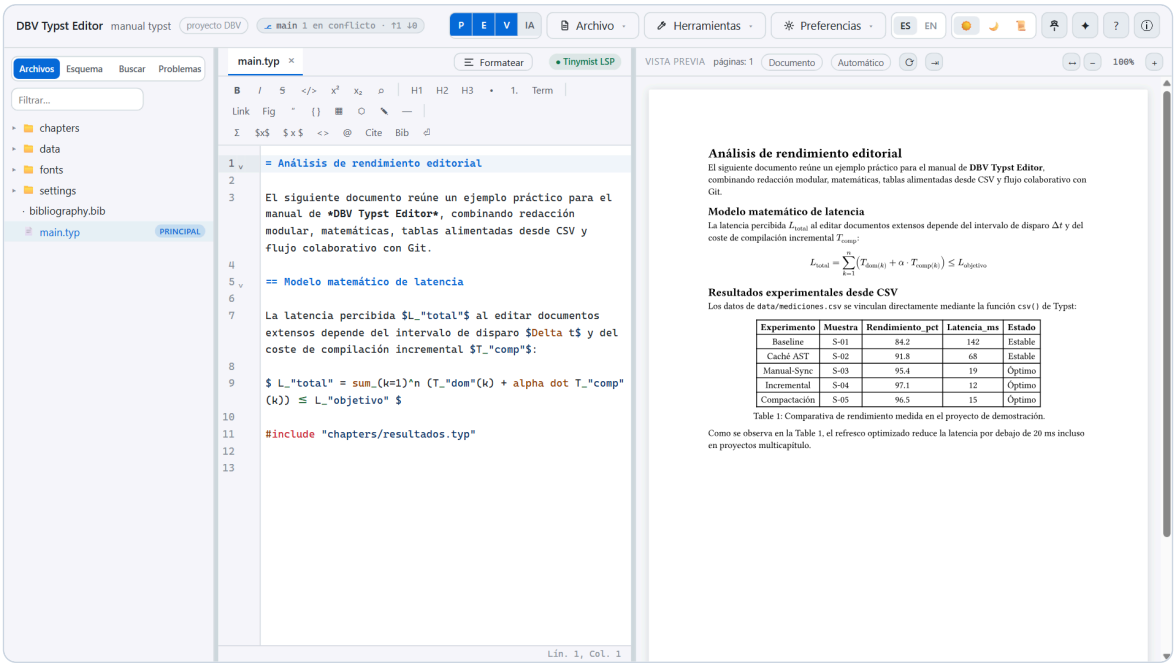


Figura 1: Vista principal dividida de DBV Typst Editor v0.15.0: árbol de proyecto, editor con sintaxis Typst y vista previa SVG compuesta en tiempo real.

1.1. Novedades principales de la versión v0.15.0

- **10 temas curados y tema personalizado WCAG AA:** Oscuro (contraste de sintaxis mejorado), Claro, Sepia, Alto contraste, Nord, Dracula, Catppuccin Mocha, Catppuccin Latte, Solarized Dark y Gruvbox Dark, además de paleta editable con tres botones rápidos configurables en cabecera.
- **Refresco Manual sin latencia al teclear:** sincronización LSP agrupada y actualización incremental de pestañas para editar documentos de cientos de páginas con fluidez total.
- **Modo Vim opcional y números de línea relativos:** carga diferida de edición modal (`NORMAL`, `INSERT`, `VISUAL`, `V-LINE`, `V-BLOCK`, `REPLACE`) con `:w` para guardar y `Escape` seguro junto a diálogos flotantes.
- **Flujo colaborativo completo con Git:** popover de estado en cabecera, detección de cambios locales antes de hacer Pull, resolvedor visual de conflictos bloque a bloque y clonación directa de repositorios.
- **Asistentes científicos y de datos:** diagramas `cetz`, ecuaciones con conversor `LaTeX` vía MiTeX, secuencia `chronos`, Gantt `gantty`, Kanban `kantan`, grafos `diagraph`, visor interactivo CSV/TSV, ejecutor de scripts Python y lógica JavaScript con `jogs`.

2.

Primeros pasos y lanzador inicial

Al abrir la aplicación sin carpeta cargada aparece el **lanzador de proyectos** (Figura 2). Desde esta pantalla puedes crear un documento guiado, abrir un proyecto ya existente o incorporar material externo.

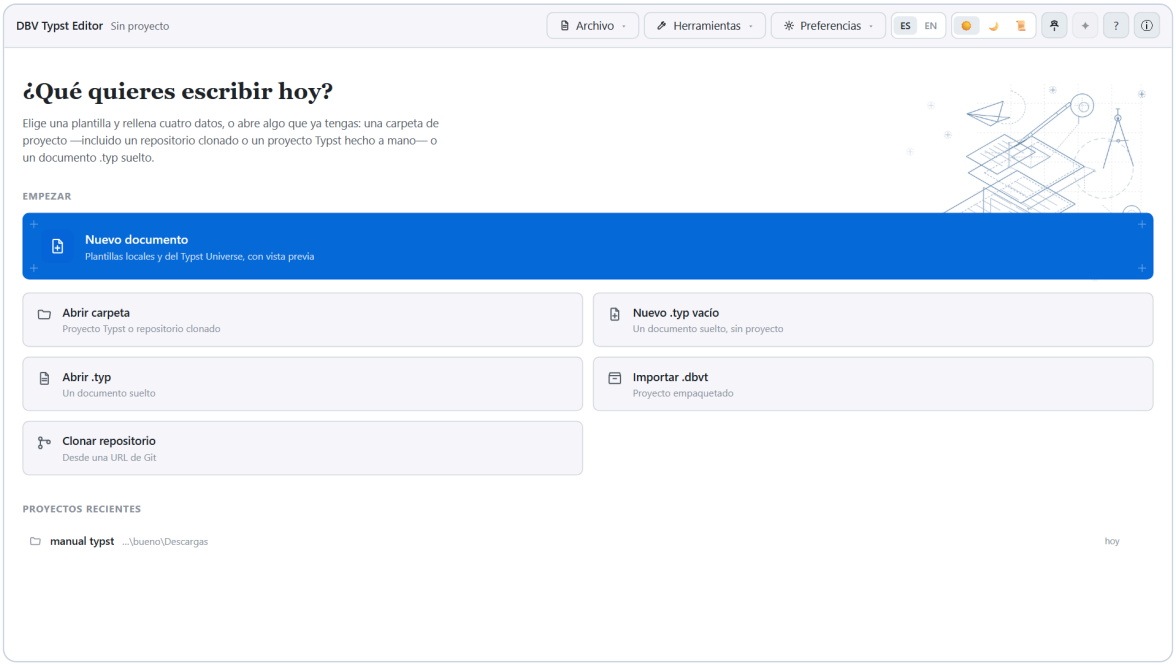


Figura 2: Lanzador inicial de DBV Typst Editor: plantillas guiadas, apertura de carpetas o ficheros sueltos, importación .dbvt y clonación de repositorios Git.

Acción del lanzador	Cuándo conviene usarla
Nuevo documento	Abre la galería con las **8 plantillas académicas propias** (Artículo científico, TFG, TFM, Tesis doctoral, Informe técnico, Presentación 16:9, Currículum vítae y Proyecto en blanco) junto al catálogo comunitario de Typst Universe. Cada plantilla propia muestra un formulario para nombre, autoría, institución, idioma y portada normalizada.
Abrir carpeta	Carga cualquier directorio que ya contenga ficheros `.typ` (por ejemplo, un repositorio Git clonado o una memoria empezada en otro ordenador). Si existe `settings/dbv-project.toml`, respeta su documento principal.
Nuevo .typ vacío	Crea un único fichero suelto fuera de proyecto para pruebas rápidas o notas matemáticas.
Abrir .typ	Edita y previsualiza un documento individual sin necesidad de carpeta de proyecto.
Importar .dbvt	Desempaqueta un archivo portable `.dbvt` recibido de un compañero o tutor, restaurando capítulos, imágenes, bibliografía y tipografías propias.
Clonar repositorio	Clona un repositorio Git remoto a una carpeta local con progreso visible y lo abre inmediatamente como proyecto listo para colaborar.

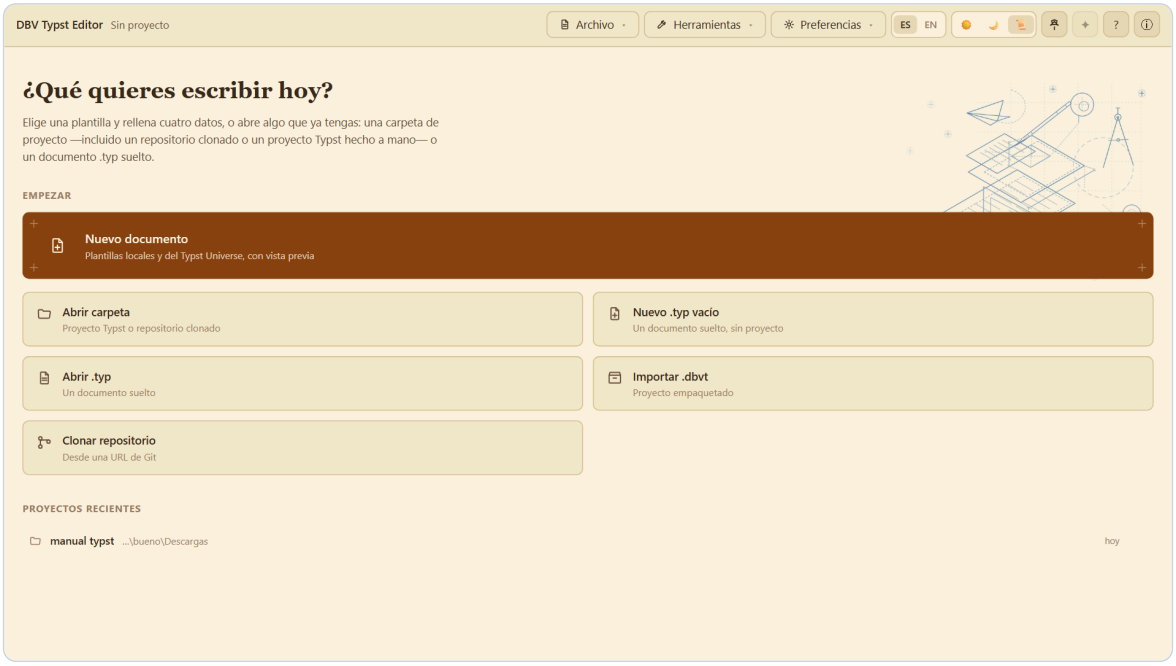


Figura 3: Lanzador con el tema Sepia activado desde los botones rápidos de la cabecera y la interfaz bilingüe ES/EN.

3.

Espacio de trabajo y los cuatro paneles

La cabecera superior concentra los conmutadores **P / E / V / IA**, los menús **Archivo**, **Herramientas** y **Preferencias**, el selector **ES / EN**, tres botones rápidos de tema, la terminal `>_`, Typst Universe ✦, la ayuda ? y **Acerca de** ℹ.

Botón	Panel y función principal
P — Proyecto	Panel lateral izquierdo con cuatro pestañas: **Archivos** (árbol del proyecto con insignia `PRINCIPAL` en el fichero raíz), **Esquema** (índice vivo de secciones), **Buscar** (búsqueda y reemplazo en todo el proyecto con regex de Rust) y **Problemas** (errores y avisos del compilador agrupados por archivo).
E — Editor	Área central de escritura con pestañas ordenables, barra de herramientas contextual, resaltado nativo Typst adaptado al tema, autocompletado LSP y barra de estado inferior (`Lín. X, Col. Y`, caracteres seleccionados e insignia de modo Vim).
V — Vista previa	Visor compuesto página a página con selector de ámbito (**Documento** completo o **Archivo** actual), refresco **Automático** o **Manual**, zoom, ajuste al ancho ↔, chincheta para sincronizar al cursor, búsqueda `Ctrl/Cmd+F` y doble clic inverso para saltar al código fuente.
IA — Asistente	Panel lateral derecho opcional. Solo se muestra cuando hay al menos un proveedor configurado y un proyecto abierto. Incluye citas automáticas de contexto, historial de hilos, árbol de propuesta de cambios y botón **Ver en vista previa**.

Los divisores entre paneles se arrastran con el ratón; un doble clic sobre el divisor devuelve el reparto por defecto. Cuando abres varios paneles flotantes a la vez (Git, Universe, Búsqueda en vista previa, Documentación o Editor de diagramas), pulsar sobre cualquiera lo trae automáticamente al frente.

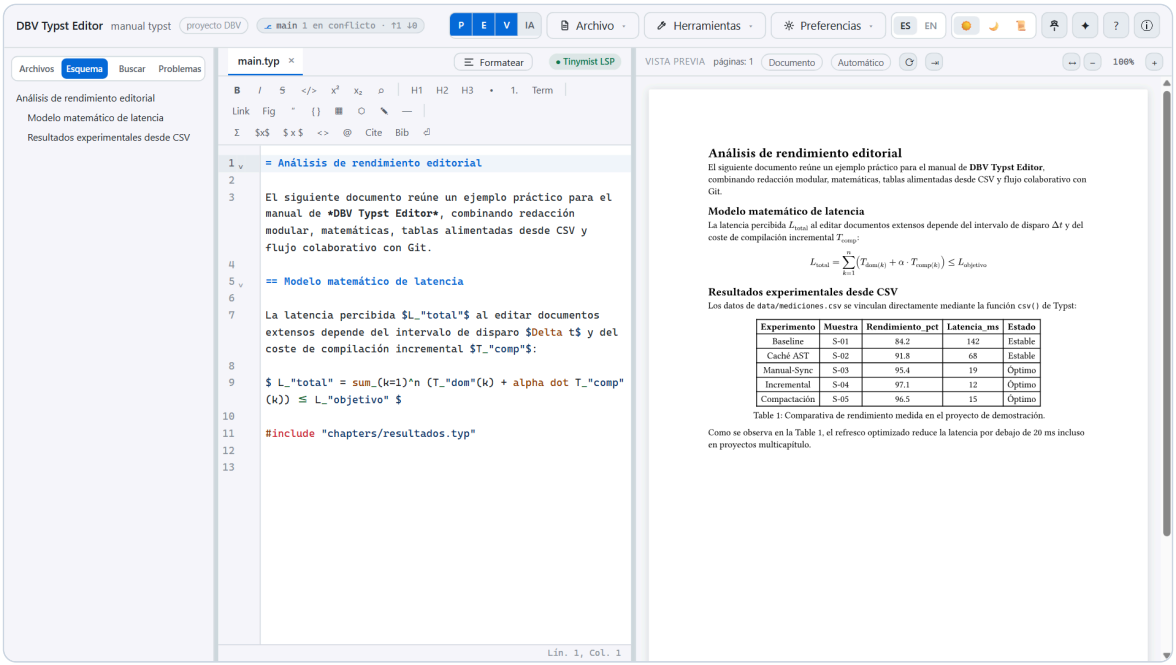


Figura 4: Pestaña Esquema mostrando la jerarquía viva de encabezados; un clic salta directamente a esa página en el documento compuesto.

4.

Referencia completa de los menús

Toda la funcionalidad de DBV Typst Editor está accesible de forma ordenada desde los tres menús de la cabecera, que alternan entre sí con un solo clic y admiten navegación completa con teclado (↑, ↓, `Inicio`, `Fin`, `Escape`).

4.1. Menú Archivo

Opción (Archivo)	Descripción y comportamiento
Nuevo proyecto en blanco	Crea una carpeta de proyecto mínima con `main.typ` listo para empezar sin formulario previo.
Nuevo documento…	Abre la galería de plantillas locales (con formulario de metadatos) y plantillas comunitarias de Typst Universe.
Guardar (`Ctrl/Cmd+S`)	Guarda el documento activo en disco. Si otro proceso o compañero modificó el fichero externamente, muestra antes el diálogo comparador lado a lado.
Guardar como… (`Ctrl/Cmd+Mayús+S`)	Guarda el archivo activo con una nueva ruta o nombre dentro o fuera del proyecto.
Exportar PDF…	Compila el documento principal actual a un archivo `.pdf` definitivo con enlaces internos, marcadores e incrustación de fuentes.
Exportar PNG…	Renderiza a imagen `.png` de alta nitidez la página visible en la vista previa (útil para figuras, portadas o diapositivas concretas).
Fijar como documento principal	Designa el fichero `.typ` actual como raíz de compilación y guarda la elección en `settings/dbv-project.toml` para todo el equipo.
Añadir capítulo…	Crea un nuevo fichero modular dentro de `chapters/` e inserta su directiva `#include` en el documento principal.
Abrir carpeta de proyecto… (`Ctrl/Cmd+O`)	Cambia a otro directorio de proyecto Typst en disco.
Abrir documento .typ…	Abre un fichero `.typ` individual sin necesidad de carpeta de proyecto.
Importar proyecto (.dbvt)…	Extrae un paquete `.dbvt` completo en la carpeta que elijas y lo abre inmediatamente.
Exportar proyecto (.dbvt)…	Empaqueta capítulos, imágenes, bibliografía, configuración `settings/dbv-project.toml` y la carpeta `fonts/` en un único archivo `.dbvt`.
Mostrar en el Explorador / Finder	Abre la carpeta del proyecto actual en el gestor de archivos del sistema operativo.
Cerrar proyecto	Cierra el proyecto activo tras comprobar cambios pendientes y vuelve al lanzador inicial.

4.2. Menú Herramientas

Opción (Herramientas)	Descripción y propósito
Conectar una IA…	Abre el asistente en tres vías: IA local (Ollama, LM Studio, OpenAI-compatible), Agentes por suscripción (Claude Code, Gemini CLI, Codex,

Opción (Herramientas)	Descripción y propósito
	Copilot CLI, Antigravity) o Nube con clave en el almacén seguro del sistema.
Aplicar plantilla…	Adapta un documento ya redactado a la estructura formal de una plantilla oficial o de Typst Universe conservando el contenido existente.
Servidor MCP…	Inicia, detiene y muestra los detalles del servidor Model Context Protocol integrado para conectar clientes externos al proyecto.
Terminal de Typst…	Abre la consola flotante integrada (`>_`) para ejecutar subcomandos directos del compilador oficial (`compile`, `fonts`, `query`, `--version`).
Ejecutar script Python…	Selecciona o ejecuta un script `.py` del proyecto (por ejemplo, generación de gráficas Matplotlib o procesado de tablas CSV) mostrando `stdout`/`stderr` y refrescando la vista previa al instante.
Editor de diagramas…	Lienzo interactivo visual para crear diagramas vectoriales exportados con el paquete `cetz` / `fletcher`.
Editor de ecuaciones…	Constructor matemático con paleta visual, vista previa real de Typst y botón **Pegar LaTeX** basado en `MiTeX`.
Diagrama de secuencia…	Asistente por participantes y mensajes ordenados en el tiempo para generar secuencias `chronos`.
Diagrama de Gantt…	Planificador de tareas con fechas reales de inicio/fin compilado mediante el paquete `gantty`.
Tablero Kanban…	Editor visual de columnas, tarjetas, responsables, dificultad y prioridad generado con `kantan` (licencia AGPL-3.0-only).
Visor de datos (CSV/TSV)…	Inspecciona ficheros tabulares, ordena y filtra columnas, e inserta tablas vinculadas mediante `csv()` o estáticas.
DOT / Graphviz…	Editor de sintaxis DOT con previsualización real de Typst mediante el paquete `diagraph`.
Editar snippets globales…	Abre tu archivo personal `snippets.code-snippets` compartido entre todos tus proyectos.
Importar snippets de Sublime…	Asistente de migración desde archivos `.sublime-snippet` o carpetas `Packages/User` hacia formato VS Code `.code-snippets`.
Editar snippets del proyecto…	Abre `.vscode/*.code-snippets` dentro del repositorio del proyecto para compartir plantillas de texto rápidas con todo el equipo.
Insertar script JS (jogs)…	Inserta una plantilla lista con el paquete `jogs` para evaluar lógica JavaScript parametrizada desde dentro del propio documento Typst.
Clonar repositorio Git…	Clona un repositorio remoto desde HTTPS o SSH directamente a una carpeta nueva y abre el proyecto resultante.

4.3. Menú Preferencias

Opción (Preferencias)	Efecto y almacenamiento
Mostrar ficheros ocultos	Muestra u oculta en el árbol lateral carpetas y archivos cuyo nombre comienza por punto (`.git`, `.vscode`, etc.) y permite incluirlos en las búsquedas globales.
Números de línea	Activa o desactiva la numeración lateral de líneas en el editor de código.
Números de línea relativos	Muestra la distancia en líneas respecto al cursor actual; especialmente práctico combinado con movimientos y operadores en modo Vim.

Opción (Preferencias)	Efecto y almacenamiento
Modo Vim	Activa o desactiva la edición modal Vim (`NORMAL`, `INSERT`, `VISUAL`, `V-LINE`, `V-BLOCK`, `REPLACE`). Desactivado por defecto para mantener el arranque instantáneo; cuando se activa muestra la insignia en la barra inferior y mapea `:w` al guardado seguro.
Guardado automático	Guarda automáticamente las pestañas modificadas tras breves pausas de edición.
Mostrar ruta completa	Visualiza la ruta completa en el título/estado del archivo activo para distinguir fácilmente capítulos homónimos.
Preguntar antes de actualizar referencias	Cuando mueves o renombras un `.typ`, `.bib` o imagen en el panel Archivos, pregunta si debe actualizar automáticamente todos los `#include`, `#import`, `image()` y `bibliography()` afectados en el proyecto.
Historial local de versiones	Conserva instantáneas automáticas antes de reemplazos masivos o cambios asistidos para poder restaurar versiones anteriores sin depender de Git.
Compartir estado con agentes MCP	Permite o bloquea que clientes externos conectados por MCP lean la posición del cursor, selección o diagnósticos activos.
Temas y colores…	Abre el panel visual con los 10 temas curados, configuración de los 3 botones rápidos de cabecera y editor completo de tema personalizado con medidor de contraste WCAG AA.
Vaciar historial local…	Limpia bajo confirmación las copias automáticas guardadas en la caché local del proyecto para liberar espacio.

5.

Edición de alto rendimiento, barra de herramientas y modo Vim

5.1. Barra de herramientas contextual y citas BibTeX

Encima del editor dispones de formato directo (**negrita**, *cursiva*, tachado, código, superíndice, subíndice), encabezados (`H1`–`H3`), listas, enlaces, notas, bloques de código, tablas e imágenes. Al pulsar dentro de una ecuación `$...$`, los botones de texto se atenúan automáticamente y pasan al frente los controles matemáticos.

Para bibliografía y referencias cruzadas:

- Escribe `@` en cualquier línea del editor para abrir el menú filtrable con las claves de figuras (`@fig:...`), tablas (`@tab:...`), ecuaciones (`@eq:...`), secciones y entradas de `bibliography.bib`.
- Pulsa **Cite** en la barra para buscar entradas por autor, título o año, o para dar de alta una nueva entrada BibTeX mediante formulario sin salir del editor.

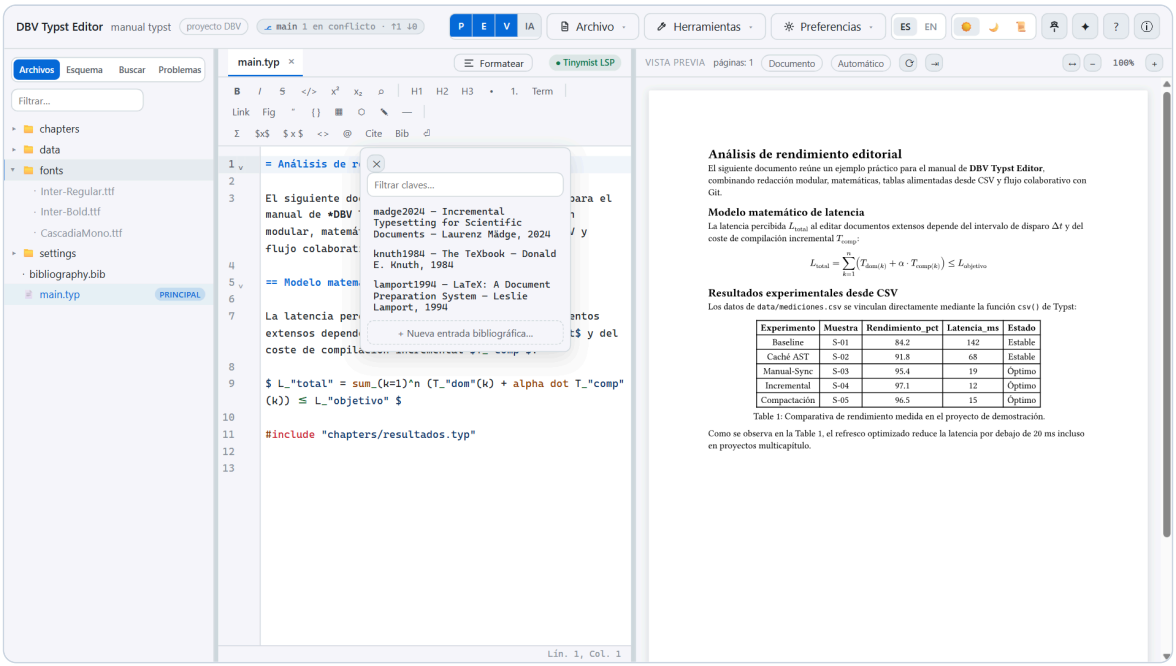


Figura 5: Desplegable interactivo de citas BibTeX desde la barra de herramientas del editor.

5.2. Modo Vim opcional, números relativos y refresco manual

Los escritores de documentos extensos y usuarios habituales de Vi/Neovim cuentan con dos herramientas complementarias en el menú **Preferencias** y la barra de vista previa:

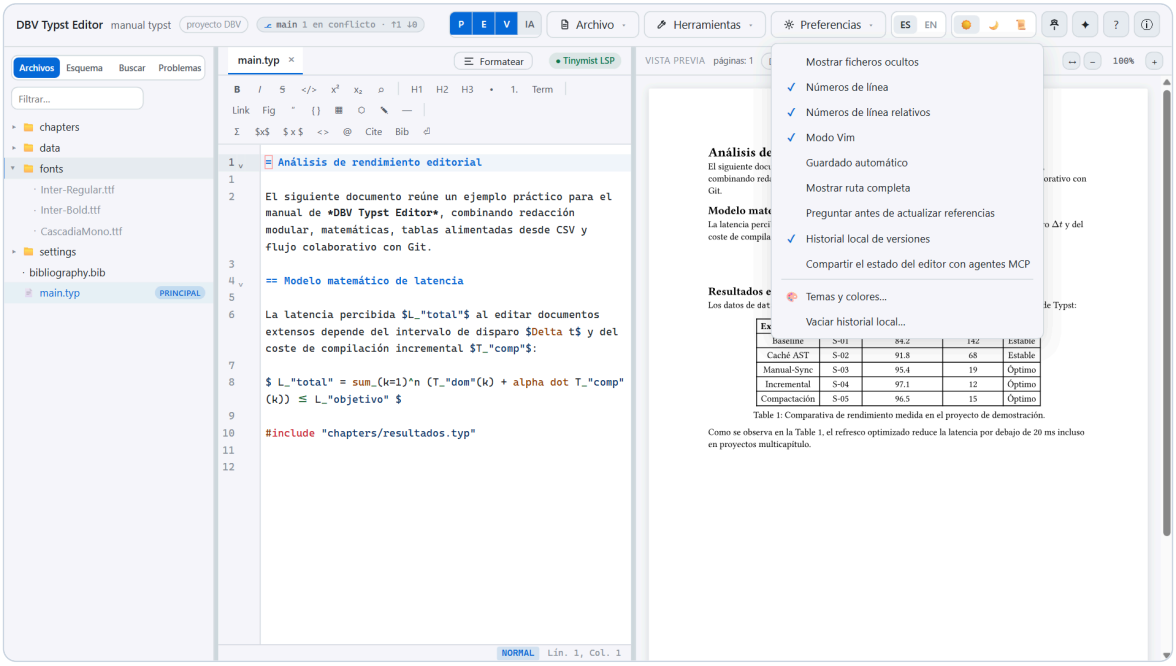


Figura 6: Menú Preferencias con Números de línea relativos y Modo Vim activados; la barra de estado inferior muestra la insignia NORMAL junto a Lín. 1, Col. 1.

1. **Modo Vim bajo demanda:** solo carga la extensión modal cuando marcas `Preferencias › Modo Vim`. Muestra en vivo `NORMAL`, `INSERT`, `VISUAL`, `V-LINE`, `V-BLOCK` o `REPLACE` en la barra de estado, mapea `:w` a guardar archivo y coordina la tecla `Escape` para regresar a modo normal sin cerrar por accidente el panel de IA ni los menús flotantes abiertos.
2. **Números de línea relativos:** facilita saltos verticales exactos (`12j`, `5k`, `d8k`) al indicar cuántas líneas separan cada fila del cursor.
3. **Refresco Manual en vista previa:** en proyectos de tesis con docenas de figuras e imágenes pesadas, cambiar el botón **Automático** a **Manual** desacopla el tecleo del compilado continuo: el LSP agrupa notificaciones y las pestañas se actualizan incrementalmente para ofrecer latencia cercana a cero al escribir; cuando quieras comprobar el resultado, pulsa el botón de recarga ⟳.

6.

Personalización visual: temas, colores y contraste WCAG

Desde `Preferencias › Temas y colores…` accedes al centro completo de personalización visual de la interfaz y de la coloración sintáctica de Typst.

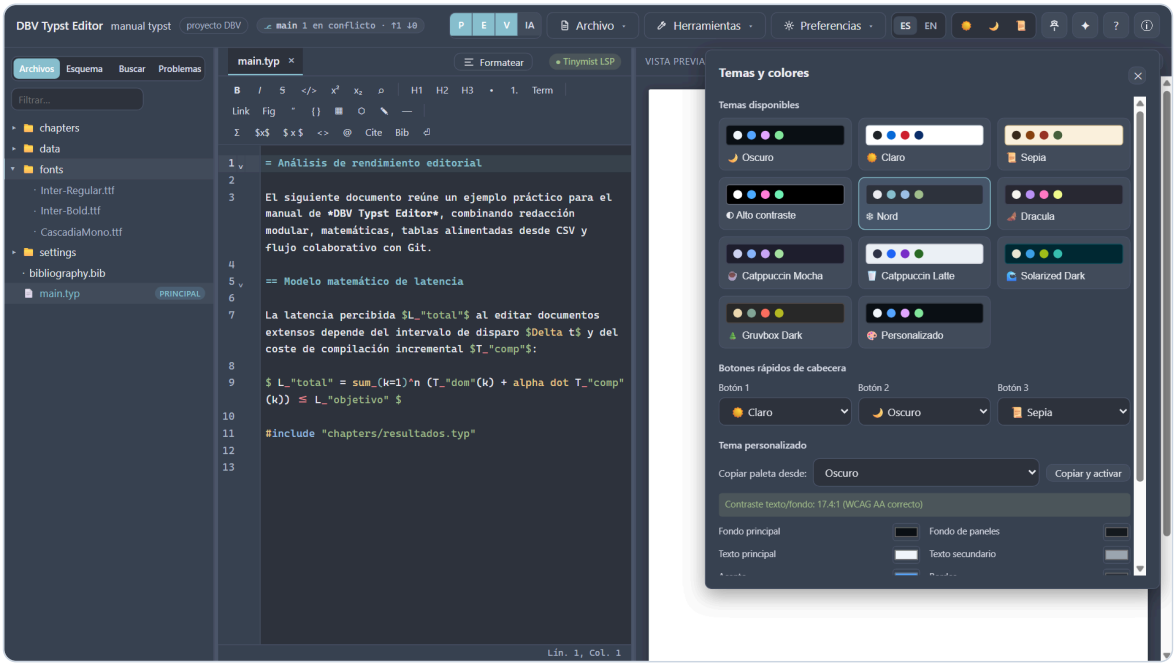


Figura 7: Diálogo Temas y colores con Nord aplicado: 10 temas curados, 3 ranuras configurables en la cabecera y ajuste personalizado con verificación WCAG AA.

- **10 paletas curadas: Claro**, **Oscuro** (con contraste mejorado para comentarios, cadenas y funciones en modo oscuro), **Sepia**, **Alto contraste**, **Nord**, **Dracula**, **Catppuccin Mocha**, **Catppuccin Latte**, **Solarized Dark** y **Gruvbox Dark**.
- **3 botones rápidos en cabecera:** asigna tus tres temas favoritos a **Botón 1**, **Botón 2** y **Botón 3** para alternar con un clic entre tema de día, noche o lectura cálida.
- **Tema personalizado y contraste WCAG AA:** pulsa **Copiar paleta desde…** para partir de cualquiera de los 10 temas oficiales y ajusta individualmente fondo principal, fondo de paneles, texto principal/secundario, color de acento, bordes y cada token sintáctico Typst mientras un indicador comprueba en vivo la relación de contraste mínima recomendada (`WCAG AA`).

7.

Trabajo colaborativo con Git

DBV Typst Editor integra un flujo Git práctico y visual pensado para coautores de artículos, grupos de investigación y directores de tesis que comparten un repositorio común.

7.1. Clonar e iniciar un proyecto compartido

- Desde el lanzador o desde `Herramientas › Clonar repositorio Git…`, introduce la URL del repositorio remoto (HTTPS o SSH) y la carpeta destino. Si el repositorio es muy pesado, puedes limitar el historial inicial mediante clonación superficial.
- Cuando trabajes en equipo, marca siempre el archivo maestro con `Archivo › Fijar como documento principal`. Eso crea o actualiza el manifiesto `settings/dbv-project.toml` para que todo compañero que clone el repositorio abra directamente la misma portada y configuración sin compilaciones fallidas sobre capítulos sueltos.
- Al renombrar o mover capítulos o figuras en el panel **Archivos**, activa `Preferencias › Preguntar antes de actualizar referencias` para reescribir de una vez todos los `#include`, `#import`, `image()` y `bibliography()`.

7.2. Estado remoto y popover de Git en cabecera

Cuando el proyecto abierto contiene un repositorio `.git`, la cabecera muestra una píldora interactiva con:

- El nombre de la **rama actual** (por ejemplo, `main`).
- El número de archivos en conflicto si hubiera una fusión pendiente (`1 en conflicto`).
- Los contadores de sincronización respecto al remoto (`↑N` commits listos para subir / `↓N` commits nuevos por descargar).

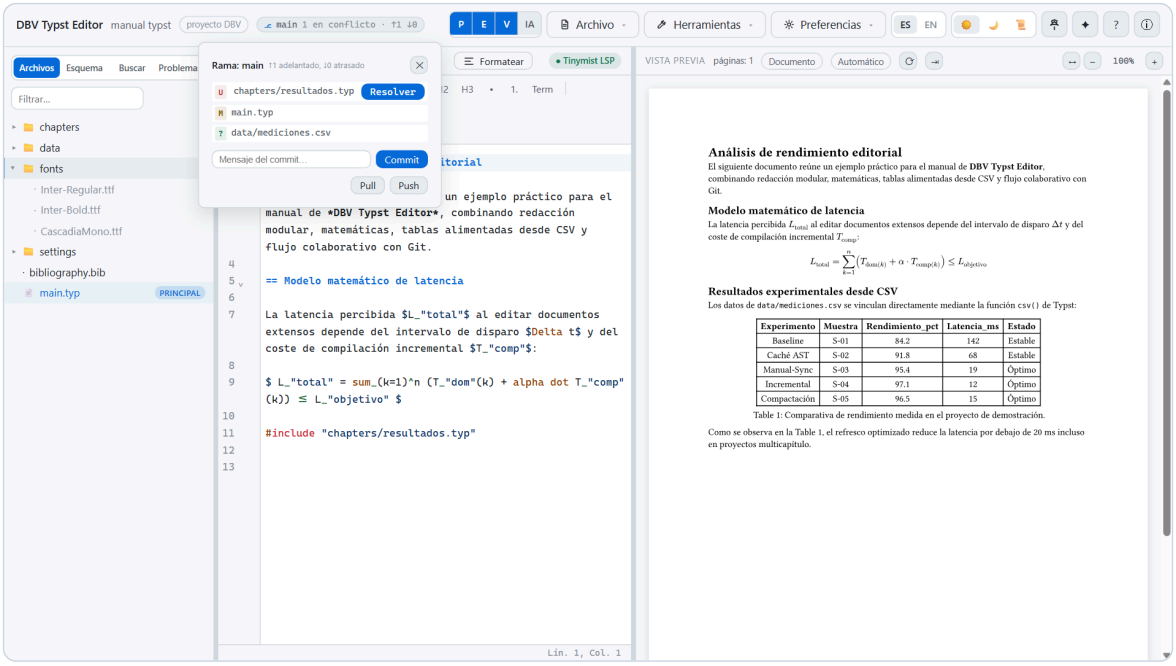


Figura 8: Popover de Git desplegado desde la cabecera: fichero en conflicto (U) con acceso directo a Resolver, archivo modificado (M), archivo nuevo sin seguimiento (?) y botones Commit / Pull / Push.

Dentro del popover cada fichero muestra su insignia de estado:

- `U` (botón azul **Resolver** en primera posición): archivo con marcas de conflicto tras un Pull o merge.
- `M`: archivo existente modificado en local.
- `?`: archivo nuevo todavía no seguido por Git.

Escribe un mensaje descriptivo en **Mensaje del commit…** y pulsa **Commit** para registrar los cambios en local, o usa **Pull** / **Push** para sincronizar con el equipo.

Protección frente a sobrescrituras en Pull

Si pulsas **Pull** mientras tienes cambios locales sin guardar en commit en archivos que también cambiaron en el servidor remoto, DBV detiene el Pull antes de mezclar nada y te avisa de que debes registrar antes tu commit local para evitar perder trabajo.

7.3. Resolvedor visual de conflictos bloque a bloque y cambios en disco

Cuando un **Pull** o una fusión deja algún archivo con marcas de conflicto (<<<<<<<, =======, >>>>>>>), no hace falta editar las marcas a mano con riesgo de romper la sintaxis Typst:

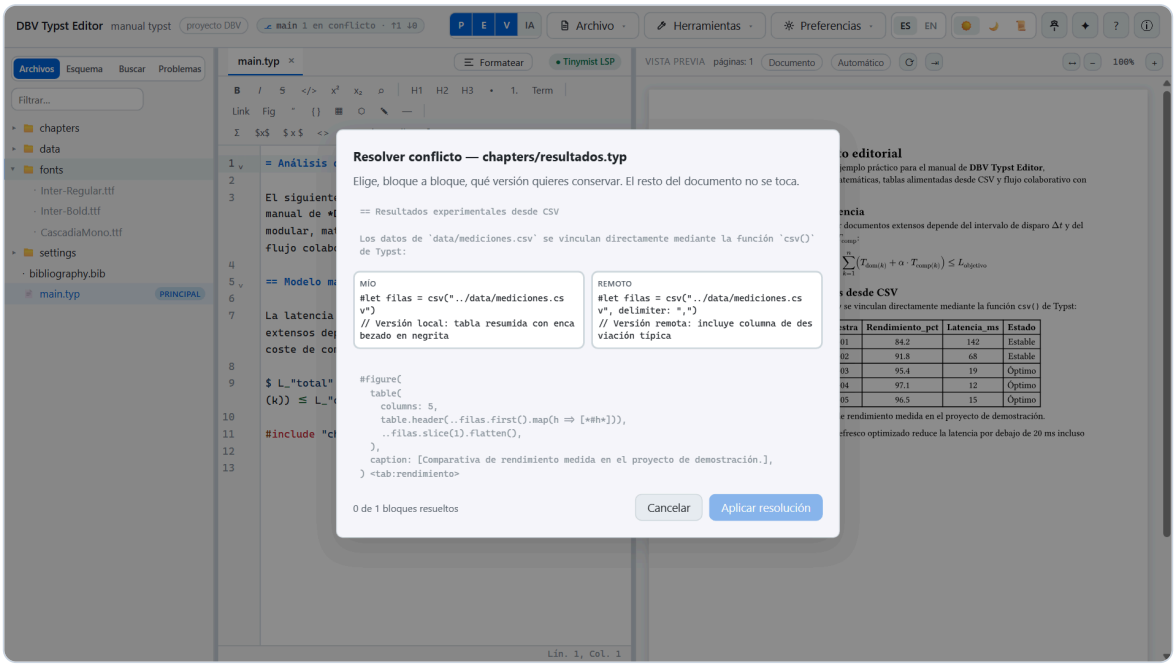


Figura 9: Resolvedor visual bloque a bloque: el usuario compara Mío frente a Remoto con el contexto intacto antes de pulsar Aplicar resolución.

1. Pulsa el botón azul **Resolver** junto al archivo marcado con `U` en el popover de Git.
2. El diálogo extrae exclusivamente cada bloque conflictivo dejando intacto el resto del documento alrededor.
3. En cada bloque eliges directamente la tarjeta **Mío** (tu versión local `HEAD`) o **Remoto** (la versión entrante del compañero), y ves en tiempo real el contador `N de M bloques resueltos`.
4. Al completar todos los bloques, pulsa **Aplicar resolución** para limpiar las marcas y dejar el fichero compilable.

De forma análoga, si un compañero o script externo modifica en disco un archivo que tienes abierto con cambios sin guardar, al guardar se abre el **comparador lado a lado** que te permite conservar tu versión del editor, recargar la versión en disco o revisar diferencias antes de sobrescribir.

8.

Herramientas visuales y asistentes con ejemplos prácticos

Todas estas utilidades viven en el menú **Herramientas** (y los principales editores rápidos también en la barra del editor) y generan código Typst limpio y editable.

8.1. Editor de diagramas (`cetz`)

Lienzo interactivo para arrastrar nodos (rectángulos, nodos redondeados, elipses, rombos de decisión, triángulos y hexágonos), conectores con flecha, seis colores semánticos, zoom con rueda y plantillas de siembra (**Flujo**, **Bloques**, **Flujo con decisión**).

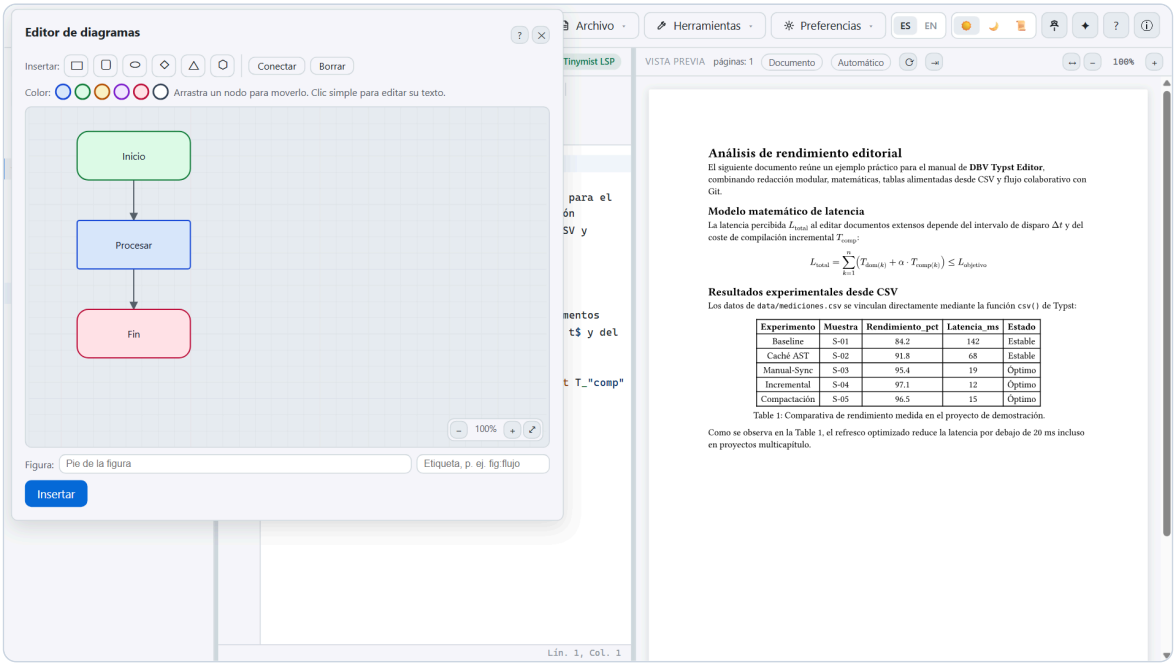


Figura 10: Editor visual de diagramas inicializado con una plantilla de flujo lista para personalizar, etiquetar e insertar en Typst.

Al pulsar **Insertar**, genera automáticamente la importación y el bloque reproducible en Typst:

```
#import "@preview/cetz:0.5.2"
#figure(
  cetz.canvas({
    import cetz.draw: *
    // Nodos y conexiones generados visualmente
    rect((0, 0), (3, 1), name: "inicio", fill: rgb("#dcfce7"))
    content("inicio", [Inicio])
  }),
  caption: [Flujo de validación editorial.],
) <fig:flujo>
```

8.2. Editor visual de ecuaciones y pegado desde LaTeX (`MiTeX`)

Incluye piezas estructurales (fracción, raíz, sumatorio, integral, matriz, griegas, delimitadores) con **vista previa real compilada por Typst** debajo del campo de código. Si ya tienes fórmulas de un artículo anterior en LaTeX, pulsa **Pegar LaTeX** para traducirlas con `MiTeX`:

```
// Ejemplo insertado o convertido desde LaTeX a ecuación numerada de Typst:
$ L_"total" = sum_(k=1)^n (T_"dom"(k) + alpha dot T_"comp"(k)) <= L_"objetivo" $
<eq:latencia>
```

8.3. Diagramas de secuencia (`chronos`)

Añade participantes y ordena mensajes síncronos (->), retornos discontinuos (-->) y notas temporales sin preocuparte por coordenadas manuales. Código Typst generado:

```
#import "@preview/chronos:0.2.1"
#figure(
  chronos.diagram({
    import chronos: *
    _par("Editor"); _par("LSP"); _par("Typst")
    _seq("Editor", "LSP", "didChange (agrupado)")
    _seq("LSP", "Typst", "compilar incremental", dashed: true)
  }),
  caption: [Secuencia de compilación incremental sin bloqueo del teclado.],
) <fig:secuencia>
```

8.4. Diagramas de Gantt (`gantty`)

Define tareas por nombre y fechas reales de inicio/fin para memorias técnicas o memorias de proyecto (TFG/TFM):

```
#import "@preview/gantty:0.5.1"
#gantty.gantt((
  tasks: (
    (name: "Especificación SDD", start: "2026-09-01", end: "2026-09-18"),
    (name: "Implementación y tests", start: "2026-09-19", end: "2026-10-15"),
    (name: "Redacción del manual", start: "2026-10-16", end: "2026-11-10"),
  )
))
```

8.5. Tableros Kanban (`kantan`)

Organiza columnas, tareas, responsables, dificultad y prioridad. Al insertarlo utiliza el paquete comunitario `kantan` (recuerda revisar su licencia `AGPL-3.0-only` en la tarjeta del diálogo).

```
#import "@preview/kantan:0.1.0": kanban
#kanban(
  columns: (
    (title: "Pendiente", cards: ((title: "Capturas EN", assignee: "Equipo"),)),
    (title: "En revisión", cards: ((title: "Manual PDF", priority: "Alta"),)),
  )
)
```

8.6. DOT / Graphviz (`diagraph`)

Permite escribir o pegar cualquier grafo `.dot` exportado de otras herramientas con vista previa compilada en vivo por Typst:

````
#import "@preview/diagraph:0.3.1": raw-render
#raw-render(```dot
digraph flujo {
  rankdir=LR;
  Editor -> Compilador [label="AST"];
  Compilador -> VistaPrevia [label="SVG"];
}
```)
````

8.7. Visor de datos CSV / TSV y tablas dinámicas (`csv()`)

Abre cualquier `.csv` o `.tsv` desde el explorador o mediante `Herramientas › Visor de datos…` para inspeccionar columnas, ordenar numéricamente o por texto, filtrar filas y elegir entre dos modos de inserción:
````

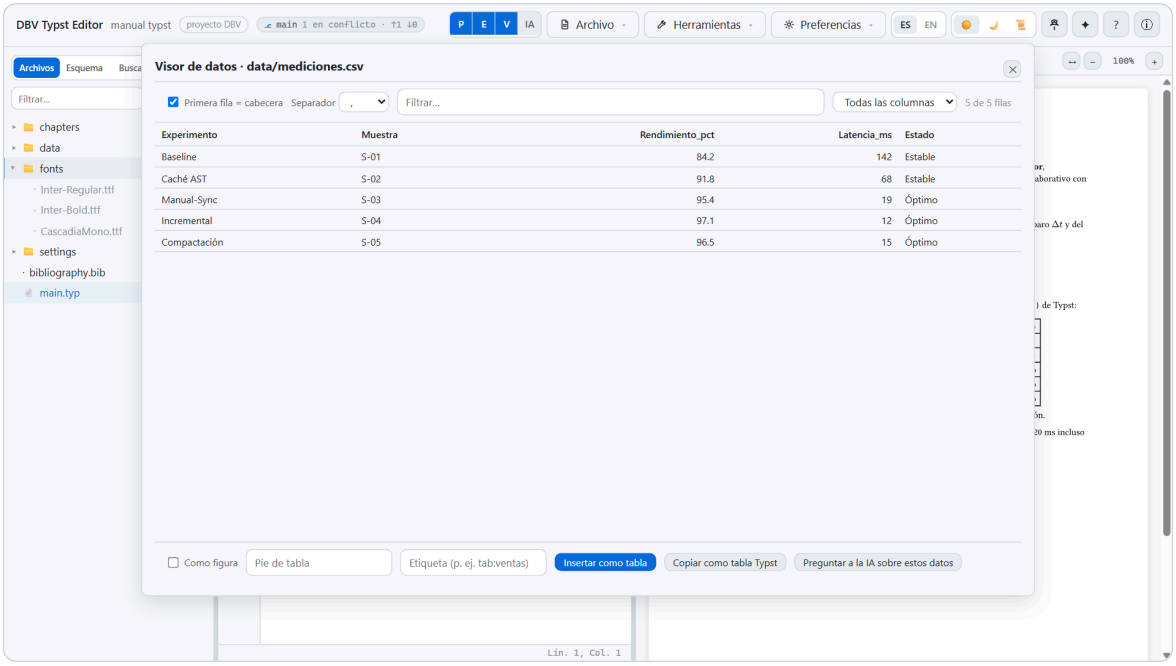


Figura 11: Visor de datos CSV/TSV con filtrado por columnas, ordenación numérica y acciones para insertar tabla viva csv() o preguntarle directamente a la IA.

- **Insertar como tabla (recomendado en ciencia e ingeniería):** inserta una tabla que lee dinámicamente el CSV con `csv()`. Cuando actualices tus experimentos o el script Python vuelva a escribir el CSV, la tabla del PDF se regenera sola en el siguiente compilado:

```
#let filas = csv("data/mediciones.csv")
#figure(
  table(
    columns: 5,
    table.header(..filas.first().map(h => [*#h*])),
    ..filas.slice(1).flatten(),
  ),
  caption: [Comparativa de rendimiento generada desde CSV.],
) <tab:rendimiento>
```

- **Copiar como tabla Typst:** convierte los datos visibles en una tabla estática literal por si deseas editar celdas manualmente o desvincularte del archivo externo.
- **Preguntar a la IA sobre estos datos:** envía una vista resumida del conjunto tabular al asistente para pedirle resúmenes estadísticos, limpieza o código Typst de visualización.

8.8. Ejecutar scripts Python y lógica JavaScript embebida (`jogs`)

- **`Herramientas › Ejecutar script Python…`:** pensado para flujos reproducibles de laboratorio y ciencia de datos. Selecciona un archivo `.py` de tu proyecto (por ejemplo un script que genera figuras PNG/SVG con `matplotlib` en `images/` o procesa un CSV en `data/`), consulta su salida estándar y errores en el diálogo integrado y comprueba al instante la actualización en el PDF sin abrir otra terminal.
- **`Herramientas › Insertar script JS (jogs)`:** añade la importación de `@preview/jogs` y un esqueleto parametrizado para evaluar pequeñas funciones JavaScript deterministas directamente dentro del documento Typst (cálculos estadísticos ligeros, formateo de series numéricas o validaciones al vuelo).

9.

Snippets de VS Code, Sublime Text y atajos personalizados

El editor comparte compatibilidad total con los archivos `.vscode/*.code-snippets` y `.vscode/keybindings.json` de VS Code para que todo el equipo use los mismos fragmentos parametrizados:

- **Ámbitos:** usa `Herramientas › Editar snippets globales…` para tus atajos personales en todos los proyectos, o `Herramientas › Editar snippets del proyecto…` (`.vscode/*.code-snippets`) para compartirlos en Git.
- **Menú contextual y selección:** si añades `"contextMenu": true` en un snippet, aparecerá al hacer clic derecho bajo **Insertar snippet ›** y envolverá el texto seleccionado mediante `$TM_SELECTED_TEXT`.
- **Importación desde Sublime Text:** `Herramientas › Importar snippets de Sublime…` lee ficheros `.sublime-snippet` individuales o carpetas `Packages/User` y genera su equivalente en JSONC mostrando una vista previa antes de guardar.
- **Teclas rápidas en `.vscode/keybindings.json`:** vincula cualquier snippet por nombre:

```
[
  {
    "key": "ctrl+alt+n",
    "command": "editor.action.insertSnippet",
    "args": { "name": "Teorema" }
  }
]
```

10.

Fuentes propias, Typst Universe y Terminal avanzado

10.1. Fuentes propias en `fonts/`

Colocar archivos `.ttf` o `.otf` dentro de una carpeta `fonts/` en la raíz del proyecto (o arrastrarlos sobre el editor cuando falta una fuente) evita tener que instalar tipografías en cada equipo: se recogen en la compilación siguiente al instante y viajan empaquetadas dentro del `.dbvt`.

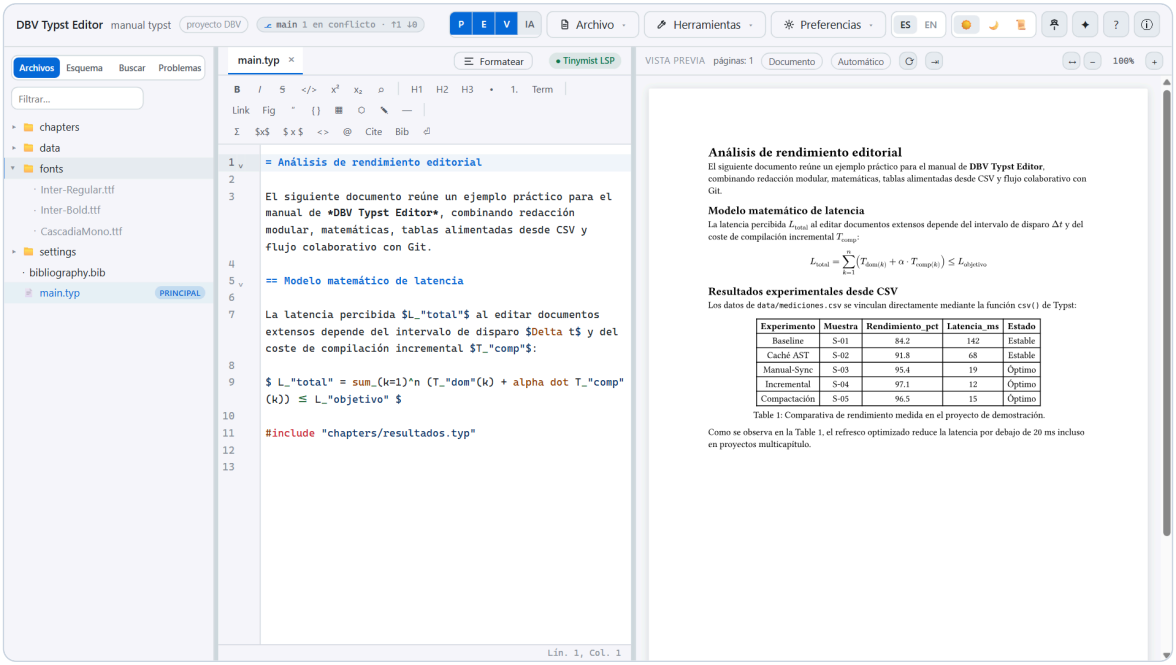


Figura 12: Carpeta fonts/ integrada en el árbol de archivos junto a main.typ (marcado PRINCIPAL), chapters/, data/ y settings/.

10.2. Typst Universe: plantillas y paquetes

El botón ✦ de la cabecera abre el catálogo verificado de plantillas científicas internacionales (IEEE, ACM, Springer, libros y diapositivas `touying`) y paquetes populares (`cetz`, `fletcher`, `codly`, `tablem`). También permite introducir cualquier identificador directo como `@preview/cetz:0.5.2` y muestra siempre la licencia antes de añadir la importación al principio del fichero.

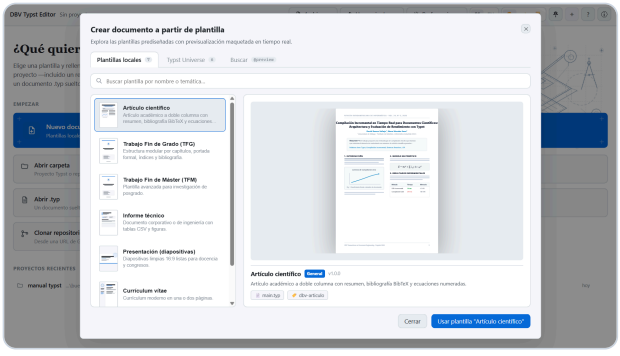


Figura 13: Galería de plantillas locales y comunitarias de Typst Universe.

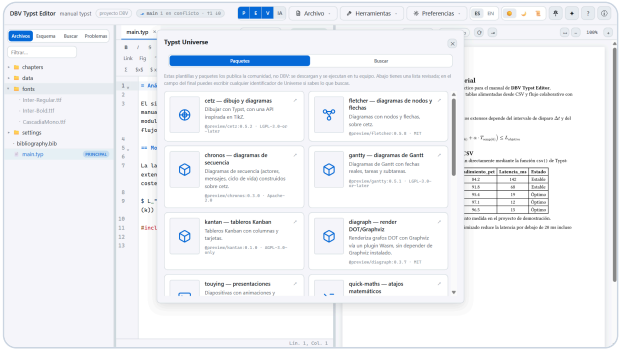


Figura 14: Catálogo de paquetes Typst Universe con inserción automática de importaciones y licencia visible.

10.3. Terminal integrado de Typst y servidor MCP

El botón >_ de la cabecera (`Herramientas › Terminal de Typst…`) da acceso directo al compilador oficial incluido en la aplicación (`compile`, `fonts`, `query`, `--version`) sobre el proyecto actual, sin necesidad de instalar Typst por separado.

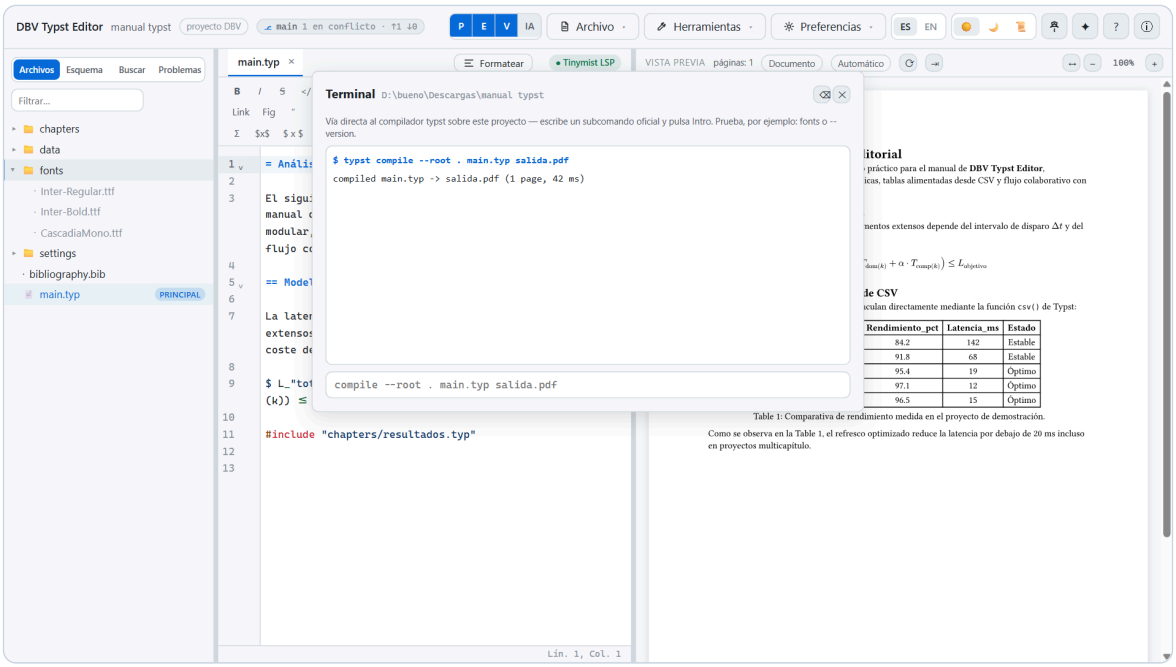


Figura 15: Terminal flotante integrado ejecutando directamente un subcomando compile del binario oficial Typst sobre el proyecto abierto.

Por su parte, `Herramientas › Servidor MCP…` arranca un punto de enlace local conforme al protocolo **Model Context Protocol** para que agentes externos inspeccionen o compilen el proyecto bajo tu control directo (`Preferencias › Compartir estado con agentes MCP`).

11.

Guía completa del Asistente de IA (Local, Agentes o Nube)

La integración con Inteligencia Artificial es completamente **opcional**: sin configurar nada, DBV Typst Editor funciona al 100 % en local. Desde `Herramientas › Conectar una IA…` el asistente detecta lo que ya tienes instalado (solo al abrir esa pantalla, nunca en el arranque) y organiza la configuración en tres vías claras.

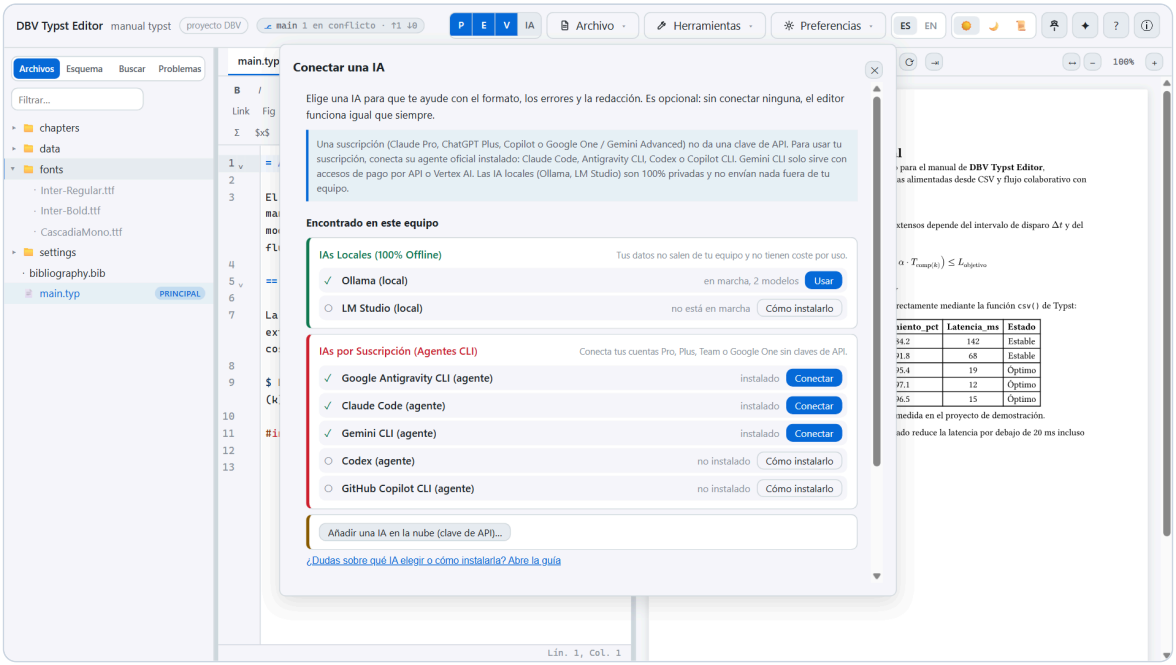


Figura 16: Asistente Conectar una IA agrupado en tres categorías: IA local (Ollama, LM Studio, servidores OpenAI-compatibles), Agentes por suscripción y Nube por clave API.

Vía de conexión	Qué necesito instalar o tener	¿Sale algo del PC?	Coste
IA local (Ollama, LM Studio, `llama.cpp`)	Servidor en marcha y al menos un modelo descargado.	No (100 % local)	Gratis (CPU/GPU)
API en la nube (Claude, OpenAI, Gemini, OpenRouter)	Una clave de API emitida por el proveedor.	Sí, contexto enviado al proveedor	Por uso (tarifa API)
Suscripción (Agentes CLI vía ACP)	Agente instalado con sesión abierta: Claude Code, Codex, Gemini CLI, Google Antigravity o GitHub Copilot CLI.	Sí, a través del agente	Incluido en tu suscripción

Una suscripción NO es una clave de API

Pagar Claude Pro/Max, ChatGPT Plus o Gemini no otorga una clave de API (se contratan aparte en la consola del proveedor). Si ya pagas una suscripción, conecta mediante los **Agentes CLI** de la tercera vía: DBV nunca ve ni guarda tus credenciales.

11.1. Vía 1: IA local en tu equipo (privacidad total)

- **Ollama (`http://localhost:11434`):** instala Ollama, descarga un modelo (`ollama pull qwen3:8b`, consulta con `ollama list` / `ollama ps`) y pulsa **Usar** en Conectar una IA. DBV habla con su API nativa (`/api/chat`) y ajusta el contexto por ti para evitar recortes silenciosos; además permite activar o desactivar el interruptor **Razonamiento**.

- **LM Studio (`http://localhost:1234`):** arranca **Developer / Local Server**; el contexto se fija en LM Studio (**Context Length**) al cargar el modelo y puedes listar sus IDs con `curl http://localhost:1234/v1/models`.
- **Tu propio servidor compatible con OpenAI (`llama-server`, vLLM, Jan…):** pulsa **Añadir una IA en la nube (clave de API)…** y elige **Servidor compatible con OpenAI (local o propio)** (no elijas **Ollama** si no lo es, o no entenderá el protocolo). Rellena la dirección terminada en `/v1` (`http://127.0.0.1:8080/v1`), el `id` devuelto por `/v1/models` y el contexto real (`n_ctx / total_slots` desde `/props`). En PowerShell usa `curl.exe` en lugar de `curl`.

Tres detalles clave al lanzar `llama-server`:

1. El contexto real es tu flag `-c`, **no** `n_ctx_train`.
2. Con múltiples ranuras paralelas (`-np`), el contexto se divide (`-c 32768 -np 4` = 8 192 por petición); usa `-np 1` para dedicarlo entero.
3. Los modelos pensadores que escriben `reasoning_content` antes que `content` pueden parecer mudos; en `llama-server` añade `--reasoning-budget 0` (ejemplo: `llama-server -m modelo.gguf -c 32768 -np 1 --reasoning-budget 0 --port 8080`).

Expectativas realistas en local: 4 096 tokens es el mínimo justo, 8 192 resulta cómodo y 32 768 holgado. Los modelos muy pequeños (2–3 B) suelen fallar proponiendo ediciones complejas; incluso con `qwen3:8b` (el mejor local pequeño evaluado) conviene revisar siempre las propuestas.

11.2. Vía 2: Proveedores en la nube con clave de API

Proveedor	Consola de claves	Endpoint preconfigurado
Anthropic (Claude)	Consola Anthropic	`https://api.anthropic.com/v1`
OpenAI (ChatGPT)	Plataforma OpenAI	`https://api.openai.com/v1`
Google (Gemini)	Google AI Studio	`https://generativelanguage.googleapis.com/v1beta/openai`
OpenRouter	openrouter.ai	`https://openrouter.ai/api/v1`

La clave se guarda exclusivamente en el **almacén cifrado de credenciales del sistema operativo** (Administrador de credenciales de Windows, Llavero de macOS o Secret Service en Linux), nunca en archivos ni `localStorage`. Al enviarse por primera vez en un proyecto, DBV pide confirmación explícita.

11.3. Vía 3: Agentes CLI con tu suscripción (protocolo ACP)

Agente CLI	Requisitos e instalación	Comando ACP
Claude Code	Claude Code en `PATH` + Node.js (`npx`). Iniciar sesión una vez en terminal.	`npx -y @agentclientprotocol/claude-agent-acp`
Codex (OpenAI)	Codex en `PATH` + Node.js (`npx`). Iniciar sesión con ChatGPT.	`npx -y @agentclientprotocol/codex-acp`
Gemini CLI	Gemini CLI instalado e identificado en terminal.	`gemini --acp`
Google Antigravity	CLI `agy` + servidor oficial `agy_acp_server` descomprimido en `%LOCALAPPDATA%\agy\acp-server` (Windows) o `~/.local/share/agy-acp-server` (macOS/Linux).	`agy_acp_server`
GitHub Copilot CLI	CLI real (`winget install GitHub.Copilot`, `brew install --cask copilot-cli` o `npm install -g @github/copilot`). El lanzador `copilot` de VS Code no basta.	`copilot --acp`

Una vez conectado un agente, el selector ⚙ permite elegir **modelo** y **nivel de razonamiento**, y 🔄 recarga la lista real de modelos disponibles. Cada conversación mantiene su propia sesión ACP en

la carpeta del proyecto. Antes de cada turno, DBV crea un **punto de restauración**: cualquier permiso (leer, editar o ejecutar) pide aprobación (**Permitir una vez**, **Siempre en esta conversación** o **Denegar**) y las ediciones directas en disco se muestran con botón **Deshacer**.

11.4. Qué puede hacer la IA sobre tu proyecto y propuestas verificadas

- **Herramientas sobre Typst:** lee y busca solo dentro de la carpeta del proyecto; consulta la documentación oficial offline de Typst 0.15.1; comprueba cada propuesta compilando en memoria; busca versiones reales y compatibles en Typst Universe sin inventar nombres; cita claves reales de tu `.bib`; inspecciona hasta 3 páginas renderizadas (si el modelo admite visión); y sugiere fuentes faltantes mostrando licencia antes de copiarlas a `fonts/`.
- **Cambio de formato («Aplicar plantilla…»):** disponible tanto sin IA (vía mecánica desde `Herramientas › Aplicar plantilla…` que traslada título, autoría, resumen y palabras clave preservando el resto y avisando de choques `#set`/`#show`) como asistido por IA (**Que la IA lo adapte**).
- **Revisión visual antes de escribir:** la tarjeta de cambios propuestos muestra si compila sin errores nuevos (`✓ Compila sin errores nuevos`) junto a **Aplicar lo marcado**, **Rechazar** y **Ver en vista previa**.

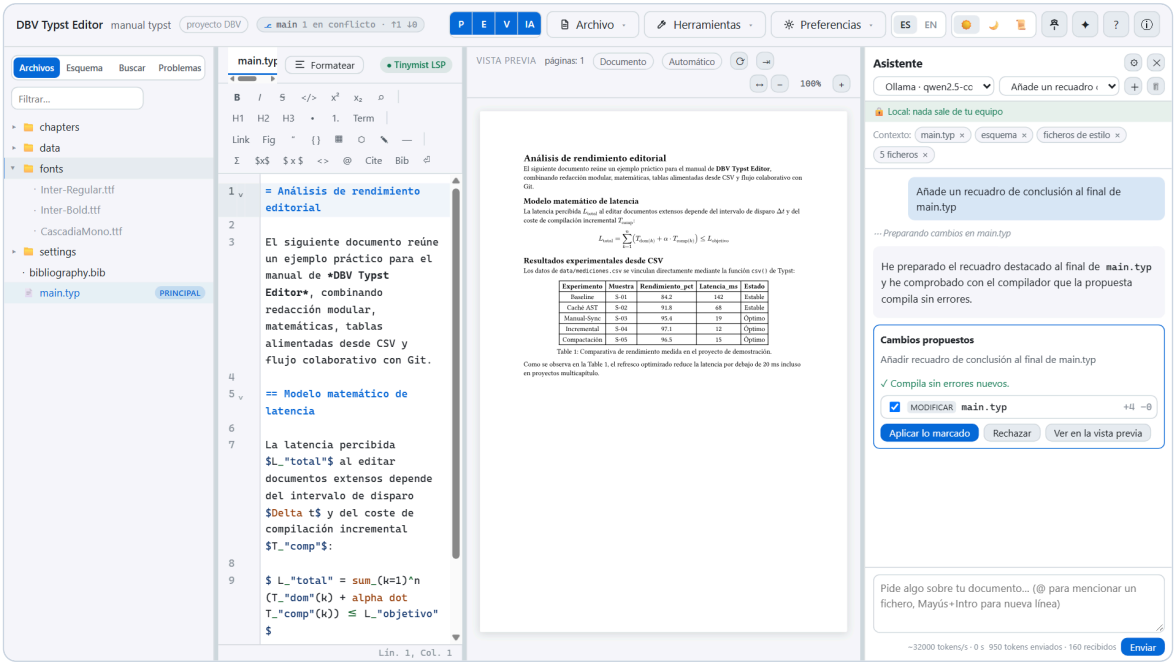


Figura 17: Panel del Asistente de IA con el árbol de propuesta de cambios, verificación previa de compilación sin errores nuevos y controles Aplicar / Rechazar / Ver en vista previa.

11.5. Privacidad y solución rápida de problemas de IA

Síntoma frecuente	Solución recomendada
No responde nada en servidor propio	Cambia el proveedor de **Ollama** a **Servidor compatible con OpenAI** y acaba la URL en `/v1`.
Respuesta vacía tras consumir tokens	El modelo está razonando; desactiva **Razonamiento** o inicia `llama-server` con `--reasoning-budget 0`.
Copilot o Antigravity no conectan	Copilot exige la CLI real (no solo VS Code); Antigravity exige `agy_acp_server` además de `agy`.
El modelo olvida parte del texto	Sube `-c` / **Contexto (tokens)** o reduce ranuras en paralelo a `-np 1`.

11.6. El servidor MCP de DBV para otros agentes (`Herramientas › Servidor MCP…`)

Además de alimentar automáticamente a los agentes del panel integrado, DBV incluye un servidor **Model Context Protocol** de solo lectura para agentes externos (Claude Code en terminal, Codex CLI, Claude Desktop, Cursor, etc.):

```
claude mcp add dbv -- "<ruta-de-DBV>" --mcp --project "<carpeta-del-proyecto>"
codex  mcp add dbv -- "<ruta-de-DBV>" --mcp --project "<carpeta-del-proyecto>"
```

Expone las herramientas `compile_project`, `render_page` (hasta 3 PNGs), `search_typst_docs`, `read_typst_docs`, `search_universe`, `read_package_docs`, `install_package` (siempre con diálogo de confirmación en DBV), `list_fonts`, `list_bibliography`, `citation_styles`, `editor_state` y `dbv_info`. Con `Preferencias › Compartir el estado del editor con agentes MCP` activado (apagado por defecto), aparece la insignia interactiva **Agente conectado** ⚔. Se ejecuta como proceso hijo sin puertos de red y con un testigo local aleatorio de 256 bits.

12.

Créditos y agradecimientos

12.1. Autoría y colaboradores

- **David Bueno Vallejo:** idea original, arquitectura, dirección del proyecto y desarrollo (`https://github.com/davidbuenov/dbv-typst-editor`, licencia MIT).
- **Juan Falgueras Cano:** colaborador del proyecto.
- **Alberto Corbi:** colaborador del proyecto y coautor del artículo de referencia sobre el ecosistema científico de Typst (Voynov, Corbi, López-Oliver & Gil, 2026, **IJIMAI**).

12.2. Agradecimientos de la comunidad (`v0.15.0`)

- **`@rgranc`** (GitHub Issues #3 y #4), por sus propuestas detalladas sobre apilamiento de ventanas flotantes, exclusividad de menús, atajos en snippets y flujo de exportación/gestión de archivos.
- **Víctor Buendía** (Fediscience), cuyas pruebas reales e ideas inspiraron la optimización de latencia en el **refresco manual**, la incorporación opcional del **Modo Vim**, la mejora de contraste sintáctico en modo oscuro y la galería ampliada de temas y colores.

12.3. Proyectos de código abierto que hacen posible DBV

- **Motor, lenguaje y sincronización: Typst** (Apache-2.0, sidecar oficial 0.15.1 y crates enlazados en proceso `typst`, `typst-ide`, `typst-layout`, `typst-svg`, `typst-kit`), **Hilbert Editor** (MIT, cuya sincronización clic→fuente con `Span` inspiró el mapa de vista previa) y **Tinymist** (Language Server oficial de Typst).
- **Paquetes de Typst Universe y editores visuales:** `cetz`, `MiTeX`, `Chronos`, `Gantty`, `Kantan`, `Diagraph` y `jogs`, así como los mantenedores de todas las plantillas y paquetes curados en **Typst Universe** (IEEE, ACM, Springer, `fletcher`, `touying`, `quick-maths`, `physica`, `codly`, `zebraw`, `showybox`, `tablem`, `subpar`, `lovelace`, `glossarium`, `unify`, `wordometer`, entre otros).
- **Infraestructura y componentes: Tauri v2** (plugins `shell`, `dialog`, `updater`, `process` y `single-instance`), **CodeMirror 6**, `codemirror-lang-typst`, `@replit/codemirror-vim`, **Hayagriva**, **Vite**, `jsonc-parser`, crates de Rust (`serde`, `tokio`, `notify`, `toml`, `toml_edit`, `tempfile`, `zip`, `dunce`, `base64`, `regex`, `walkdir`, `ureq`, `sys-locale`) y `tauri-windows-bundle` de **Vladimir Pankratov** (empaquetado `.msix` para Microsoft Store).
- **Construido con asistencia de IA y SDD: Gemini** (Google DeepMind), **Claude Code** (Anthropic), **Microsoft Copilot** (Microsoft) y el framework abierto **dbv-specs-ops**.